Codeforces Round 1084 (Div. 3)

A. Eating Game

显然找到多个最大值的位置即可

时间复杂度 $O(n)$。

```
void solve(){
    ll n = read();
    vector<ll> a(n);
    for(ll i=0;i<n;i++) a[i] = read();
    ll mx = *max_element(all(a));
    ll ans = 0;
    for(ll i=0;i<n;i++) ans += (a[i] == mx);
    print(ans);
}
```

B. Deletion Sort

如果原来不是单调不减的，则 AksLolCoding 可以一直删除除了最大值外的数字，直到只剩一个，答案为 1。

否则无法操作，答案为 n。

时间复杂度 $O(n)$。

```
void solve(){
    ll n = read();
    vector<ll> a(n);
    for(ll i=0;i<n;i++) a[i] = read();
    ll f = 1;
    for(ll i=1;i<n;i++) f &= (a[i] >= a[i-1]);
    if(f) print(n);
    else print(1);
}
```

C.Specialty String

观察发现有效串的定义与有效括号一致，所以用栈匹配一下即可。

时间复杂度 $O(n)$。

```
void solve(){
    ll n;cin>>n;
    string s;cin>>s;
    vector<char> stk;
    for(auto v:s){
        if(stk.size()&&stk.back()==v){
            stk.pop_back();
```

```
        }
        else{
            stk.pb(v);
        }
    }
    cout<<(stk.empty()?"YES\n":"NO\n");
}
```

D. Portal

对于原数组，不妨分为$[x,y]$ 内和 $[x,y]$ 外两部分。则所有操作等价于一下两部分：

- $[x,y]$ 内循环。 $[x\ \ 1\ \ 2\ \ 3\ \ y]->[x\ \ 2\ \ 3\ \ 1\ \ y]->[x\ \ 3\ \ 1\ \ 2\ \ y]$。
- $[x,y]$ 外则相当于改变 $[x,y]$ 的插入位置。 $[1\ \ x\ \ 2\ \ y\ \ 3]->[1\ \ 2\ \ x\ \ 3\ \ y]$。

显然 $[x,y]$ 我们可以循环到最小值在最前面。插入时插在 $[x,y]$ 外第一个 比最小值大的数字前面即可。

时间复杂度 $O(n)$。

```
void solve(){
    ll n = read(), x = read(), y = read();
    vector<ll> a(n);
    for(ll i=0;i<n;i++) a[i] = read();
    vector<ll> t1, t2;
    for(ll i=0;i<x;i++) t1.pb(a[i]);
    for(ll i=y;i<n;i++) t1.pb(a[i]);
    for(ll i=x;i<y;i++) t2.pb(a[i]);
    vector<ll> tt2;
    ll mi = *min_element(all(t2));
    ll idx;
    for(idx=0;idx<n&&t2[idx]!=mi;idx++);
    for(ll i=0;i<t2.size();i++,idx=(idx+1)%t2.size()) tt2.pb(t2[idx]);
    vector<ll> ans;
    ll f = 0;
    for(ll i=0;i<t1.size();i++){
        if(f) ans.pb(t1[i]);
        else{
            if(t1[i]<mi) ans.pb(t1[i]);
            else{
                f = 1;
                for(auto v:tt2) ans.pb(v);
                ans.pb(t1[i]);
            }
        }
    }
    if(!f){
        for(auto v:tt2) ans.pb(v);
    }
    print(ans);
}
```

E. Divisive Battle

如果原先已经不减，则 Bob 赢。

如果有任何一个数字包含至少两个不同的质因子，那么 `Alice` 总可以在第一轮操作中把最小的质因子拆出来放到后一个，创造出一个Bob 无法消除的下降序列。则 `Alice` 赢。例如 $18->9\ \ 2$

否则所有的数字都是质数的次方或者 **1** ，此时操作到最后必定会变成质数或者 1 的数组，只需判断这个数组是不是不降。如果不降 Bob 赢，否则 `Alice` 赢。

时间复杂度 $O(n)$。

```
const ll maxn = 1e6 + 7;
vector<ll> d(maxn);
vector<ll> f(maxn);// f[i] 用于标记数字是不是只有一个质因数。不是为0,是为对应的质因数
void init(){
    iota(all(d), 0ll);
    for(ll i=2;i<maxn;i++){
        if(d[i] != i) continue;
        for(ll j=i;j<maxn;j+=i){
            d[j]  = i;
        }
    }
    f[1] = 1;// 注意1的特判
    for(ll i=2;i<maxn;i++){
        if(d[i] != i) continue;
        for(ll j=i;j<maxn;j*=i){
            f[j] = i;
        }
    }
}

void solve(){
    ll n = read();
    vector<ll> a(n+1);
    ll t = 1;
    for(ll i=1;i<=n;i++){
        a[i] = read();
        if(i > 1) t &= a[i] >= a[i-1];
    }
    if(t){
        puts("Bob");
        return;
    }
    ll f1 = 0;
    for(ll i=1;i<=n;i++){
        if(f[a[i]] == 0) f1 = 1;
    }
    if(f1){
        puts("Alice");
        return;
    }
    vector<ll> b(n+1);
    for(ll i=1;i<=n;i++){
        b[i] = f[a[i]];
```

```
    }
    ll premx = 0;
    for(ll i=1;i<=n;i++){
        if(b[i]<premx){
            puts("Alice");
            return;
        }
        premx = max(premx, b[i]);
    }
    puts("Bob");
    return;
}
```

F. Mooclear Reactor 2

对于一个大小为 z 的组，我们会选择所有 $y >= z-1$ 的元素中最大的 z 个元素。

则有两种情况：

- 原先的 n 个中最大的情况，即所有大小的组中最大的那一个。
- 原先的所有大小的组中空缺最小的那一个，在加入当前元素。

y 从大到小的过程中，限制越来越严，组的大小减小。我们可以用优先队列维护所有 $y >= z-1$ 的元素，选出最大的 z 个值 ,和为 A_z。

同时维护删除一个元素给一个空位后的和 B_z。

则最终的答案为 $max\{\max_{i=0}^{n}\{A_i\}, x+\max_{i=0}^{y+1}\{B_i\}\}$。

时间复杂度 $O(nlogn)$

```
void solve(){
    ll n = read(), m = read();
    vector<PLL> memo;
    for(ll i=0;i<n;i++) memo.pb({read(), read()});
    vector<vector<ll>> t(n+2);
    for(ll i=0;i<n;i++) t[memo[i].se+1].pb(memo[i].fi);
    priority_queue<ll, vector<ll>, greater<ll>> pq;
    vector<ll> a(n+2),b(n+2);
    ll sum = 0, mx = 0;
    for(ll i=n+1;i>=1;i--){
        for(auto v:t[i]) pq.push(v),sum+=v;
        while(pq.size()>i){
            ll t = pq.top();
            sum-=t;
            pq.pop();
        }
        mx = max(mx,sum);
        a[i] = sum;
        if(pq.size()==i){
            b[i] = sum - pq.top();
        }
        else{
```

```
            b[i] = sum;
        }
    }
    for(ll i=1;i<=n+1;i++){
        a[i] = max(a[i], a[i-1]);//求前缀max
        b[i] = max(b[i], b[i-1]);
    }
    while(m--){
        ll x = read(), y = read();
        pt(max(a[n+1],b[y+1]+x)),putchar(' ');
    }
    putchar('\n');
}
```

G. Operation Permutation

显然可以归类为两种运算 $+$ 和 $\times$ 。（除法求逆元）

对于 $+$ ，其贡献取决于后面乘了多少个元素。

设有 cnt 个 $\times$ ，令 dp_k 表示从 cnt 个乘法中选择 k 个的所有方案乘积的和， sum 为所有加法算式的和。

则 dp_k 可以用类似 01 背包的方式求出。

若后面有 k 个乘法，则贡献为 $\frac{1}{cnt+1} * \frac{1}{C_{cnt}^k} * dp_k$。

$\frac{1}{cnt+1}$ 是由于后面有 $[0, cnt]$ 个乘法是等概率的，可以视为加法等概率插入到 cnt 个乘法的间隙当中。

$\frac{1}{C_{cnt}^k}$ 则是从 cnt 个乘法中选择 k 个的所有方案数。

对于初始的 x ，其固定乘 cnt 个乘法。而余下的所有加法的式子可以合并到一起。

则答案为 $x * dp_{cnt} + \frac{1}{cnt+1} * sum * \sum_{i=0}^{cnt} \frac{1}{C_{cnt}^k} * dp_k$ 。

时间复杂度 $O(n^2)$ 。瓶颈在 01 背包。

```
//Combination
class Comb{
    private:
    vector<ll> fac, invfac;
    public:
    Comb(ll n){
        fac.resize(n+1);
        invfac.resize(n+1);
        fac[0] = 1;
        for(ll i=1;i<=n;i++){
            fac[i] = (fac[i-1] * i)%MOD;
        }
        invfac[n] = ksm(fac[n], MOD-2);
        for(ll i=n-1;i>=0;i--){
            invfac[i] = (invfac[i+1] * (i+1))%MOD;
        }
    }
```

```
    ll C(ll n, ll m){
        if(n==0) return 1;
        if(n < m|| m < 0) return 0;
        return fac[n]*invfac[m]%MOD*invfac[n-m]%MOD;
    }
    ll A(ll n, ll m){
        if(n < m|| m < 0) return 0;
        return fac[n]*invfac[n-m]%MOD;
    }
};

Comb C(1e4+7);
void solve(){
    ll n,x;cin>>n>>x;
    vector<ll> mul;
    ll sum = 0;
    for(ll i=0;i<n;i++){
        char c;ll num;cin>>c>>num;
        if(c == '+') sum = (sum + num%MOD)%MOD;
        else if(c == '-') sum = (sum - num%MOD + MOD)%MOD;
        else if(c == 'x') mul.pb(num%MOD);
        else mul.pb(ksm(num));
    }
    ll cnt = mul.size();
    vector<ll> dp(cnt+1);
    dp[0] = 1;
    for(ll i=0;i<cnt;i++){
        for(ll j=i+1;j>=1;j--){
            dp[j] = (dp[j]+dp[j-1]*mul[i]%MOD)%MOD;
        }
    }
    ll ans = x * dp[cnt]%MOD;
    ll res = 0;
    for(ll i=0;i<=cnt;i++){
        res = (res + dp[i]*ksm(C.C(cnt,i))%MOD)%MOD;
    }
    ans = (ans + res*ksm(cnt+1)%MOD*sum%MOD)%MOD;
    cout<<ans<<'\n';
}
```

H. Six Seven

~~等我补了在写题解~~

C++ 火车头

```
// FZANOTFOUND
#include <bits/stdc++.h>
using namespace std;

#define pb push_back
#define eb emplace_back
```

```
#define fi first
#define se second
#define ne " -> "
#define sep "======"
#define fastio ios::sync_with_stdio(false);cin.tie(0);
#define all(a) a.begin(), a.end()

typedef long long ll;
typedef unsigned long long ull;
typedef long double db;
typedef pair<long long,long long> PLL;
typedef tuple<ll,ll,ll> TLLL;
const ll INF = (ll)2e18+9;
const ll MOD = 1000000007;
//const ll MOD = 998244353;
const db PI = 3.14159265358979323;

//io functions
inline void rd(ll &x){x=0;short f=1;char c=getchar();while((c<'0'||c>'9')&&c!='-
') c=getchar();if(c=='-') f=-1,c=getchar();while(c>='0'&&c<='9') x=x*10+c-
'0',c=getchar();x*=f;}
inline ll read(){ll x=0;short f=1;char c=getchar();while((c<'0'||c>'9')&&c!='-')
c=getchar();if(c=='-') f=-1,c=getchar();while(c>='0'&&c<='9') x=x*10+c-
'0',c=getchar();x*=f;return x;}
inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}
inline void print(ll x){pt(x), puts("");}
inline void print(PLL x){pt(x.fi), putchar(' '), pt(x.se), putchar('\n');}
inline void print(vector<ll> &vec){for(const auto t:vec)pt(t),putchar('
');puts("");}
inline void print(const map<ll, ll>& g) {for(const auto& [key, value]:g)
{cout<<"key: "<<key<<ne<<value<<" ";}puts("");}
inline void print(vector<PLL> &vec){puts(sep);for(const auto v:vec)
{print(v);}puts(sep);}
inline void print(const map<ll, vector<ll>>& g) {for (const auto& [key, value] :
g) { cout << "key: " << key << ne;for (const auto& v : value) {cout << v << "
";}cout << endl;}}

//fast pow
ll ksm(ll a, ll b=MOD-2, ll M=MOD){a%=M;ll res=1;while(b){if(b&1){res=
(res*a)%M;}a=(a*a)%M;b>>=1;}return res;}

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());//rng()
ull randint(ull l, ull r){uniform_int_distribution<unsigned long long> dist(l,
r);return dist(rng);}

void init(){

}

void solve(){

}

int main(){
    init();
```

```
    ll t = 1;
    //fastio;cin>>t;
    t = read();
    while(t--){
        solve();
    }
}
```